INFORME TÉCNICO DE DESARROLLO

NexaBank Wallet

Billetera digital móvil (PWA) — Auditoría de API, mejoras de backend y construcción del frontend desde cero

Fecha de corte del informe: 22 de octubre de 2026
Repositorios: NexaBank-Backend (nexabank-api) · NexaBank-Frontend
Elaborado con Claude Code (Anthropic), bajo supervisión y dirección del desarrollador

Índice

1. Introducción y objetivo del proyecto 3
2. Metodología de trabajo 3
3. Fase 1 — Auditoría de la API existente 4
4. Fase 2 — Mejoras aplicadas al backend 6
5. Infraestructura y control de versiones 8
6. Fase 4-5 — Scaffold y primer sistema de diseño del frontend 9
7. Corrección del flujo de autenticación 10
8. Rediseño "Obsidiana y Lima" 11
9. Verificación y aseguramiento de calidad 13
10. Estado actual y trabajo pendiente 14
11. Conclusiones 15

Resumen ejecutivo

Este informe documenta el trabajo de desarrollo realizado sobre el ecosistema NexaBank: un backend bancario existente en Spring Boot / Firebase Firestore, y la construcción desde cero de su frontend como una Progressive Web App (PWA) móvil. El trabajo se ejecutó en fases, siguiendo una especificación maestra (`WALLET_FRONTEND_SPEC.md`) que exigía auditar la API antes de escribir una sola línea de frontend, y avanzar de forma incremental y verificable en cada paso.

98/98
TESTS BACKEND PASANDO

19
RIESGOS DE API IDENTIFICADOS

9
PANTALLAS MÓVILES NAVEGABLES

2
ITERACIONES DE DISEÑO VISUAL

1. Introducción y objetivo del proyecto

NexaBank es un proyecto de billetera digital móvil que ya contaba con un backend funcional (`nexabank-api`, Spring Boot 3.2.5 sobre Java 17, con persistencia en Firebase Firestore) pero carecía de frontend. El objetivo de este trabajo fue doble: primero, verificar que la API existente fuera lo suficientemente sólida y segura como para soportar un cliente real antes de construir sobre ella; segundo, construir ese cliente — una PWA móvil, API-first, con un sistema de diseño propio — de forma completa y funcional, aunque sea en modo simulado (mock), para poder validar los flujos de usuario sin depender de que el backend estuviera terminado al 100%.

El proyecto se abordó explícitamente como un desarrollo de banca digital: esto implica que cualquier cambio relacionado con autenticación, manejo de dinero, cifrado o migraciones de datos se trató como de alto riesgo y requirió confirmación explícita antes de ejecutarse, en lugar de aplicarse de forma automática.

2. Metodología de trabajo

El desarrollo siguió ocho fases definidas en la especificación maestra del proyecto:

Fase	Objetivo	Estado al corte de este informe
1	Auditar la API Spring Boot existente	**Completa**
2	Proponer y aplicar mejoras seguras a la API	**Completa** (bloques autorizados)
3	Generar contrato OpenAPI y cliente tipado	Pendiente
4	Scaffold del frontend (Vite + PWA)	**Completa**
5	Sistema de diseño (tokens + componentes + `/dev/ui`)	**Completa** (2 iteraciones)
6	Capa de mocks (MSW)	Parcial — mocks en memoria, sin interceptor de red
7	Pantallas y flujos completos	Parcial — auth completo, home y transferencia completos; resto en stub
8	Calidad: tests, a11y, Lighthouse, README	Parcial — verificación manual continua, sin suite automatizada aún

Principio de verificación continua

Cada fase se cerró con evidencia observable antes de darse por terminada: en el backend, ejecución real de la suite de tests (`mvn test`); en el frontend, compilación estricta de TypeScript, arranque del servidor de desarrollo, y — de forma deliberada — navegación real de cada pantalla con un navegador Chromium headless (Playwright) para capturar evidencia visual y detectar errores de consola, en lugar de confiar únicamente en que el código "debería funcionar".

Trabajo delegado y orquestado

Las tareas de mayor volumen (auditoría completa de un backend multi-módulo, implementación simultánea de varios bloques de seguridad, construcción completa de un scaffold de frontend) se delegaron a agentes especializados que trabajaron en paralelo cuando no tenían dependencias entre sí — por ejemplo, la implementación de las mejoras del backend y el scaffold inicial del frontend se ejecutaron de forma simultánea una vez que la auditoría y el plan de mejoras quedaron aprobados. Cada entrega se revisó, y en el caso de cambios de alto riesgo (autenticación, dinero, PIN), se exigió confirmación explícita por bloque antes de tocar código.

3. Fase 1 — Auditoría de la API existente

Antes de escribir código de frontend, se realizó una auditoría exhaustiva y de solo lectura del backend `nexabank-api`: estructura del proyecto, cada endpoint REST, modelos de datos, seguridad, integración con Firebase, manejo de errores, validación y cobertura de tests existente. El resultado se documentó en `docs/API_AUDIT.md`.

Stack detectado

- Spring Boot 3.2.5 sobre Java 17, build Maven multi-módulo.
- Persistencia en Firebase Firestore (NoSQL) vía Firebase Admin SDK — sin base de datos relacional.
- Autenticación por Firebase ID Token, verificado contra el Admin SDK.
- Sin documentación OpenAPI/Swagger existente al momento de la auditoría.
- 613 líneas de test, con cobertura superficial concentrada en transferencias, beneficiarios y FX — sin ningún test para autenticación, cuentas, transacciones, QR ni el filtro de seguridad.

Hallazgo principal

La seguridad no estaba realmente aplicada a nivel de framework. `SecurityConfig` registraba el filtro de autenticación de Firebase, pero terminaba con `.anyRequest().permitAll()`, por lo que ningún endpoint —incluidas transferencias y rutas de administrador— exigía un token válido a nivel de Spring Security. La única protección real dependía de que cada controlador llamara manualmente a una utilidad de contexto de usuario, y varios no lo hacían.

Resumen de riesgos identificados (19 en total)

Severidad	Cantidad	Ejemplos
Alta	7	Autorización no aplicada a nivel de framework; IDOR en lectura de transacciones/transferencias por ID; mutación de balance sin transacción atómica en pagos QR; PIN almacenado y comparado en texto plano; dinero persistido como `double` en Firestore; endpoints de siembra/purga de base de datos sin autenticación; rol ADMIN sin aplicación real
Media	9	CORS con comodín combinado con credenciales habilitadas; validación `@Valid` faltante en varios endpoints; idempotencia que no valida el contenido del request; error 500 genérico que filtra el mensaje interno de la excepción; enumeración de usuarios por teléfono sin límite de tasa; código OTP logueado en texto claro
Baja	3	Generación de IDs por `Math.random()` sin verificación de unicidad; sin límite de tasa en ningún endpoint; exposición pública de `/actuator/health`

Cobertura del contrato requerido por el frontend

Se comparó el backend real contra el contrato de 24 grupos de endpoints que el frontend necesitaría (autenticación, usuario, wallet, transacciones, contactos, tarjetas, notificaciones):

3	9	12
COINCIDEN TAL CUAL (EXISTS)	EQUIVALENTES CON NAMING DISTINTO (PARTIAL)	NO EXISTEN EN EL BACKEND (MISSING)

Las ausencias completas incluyen: KYC, gestión de dispositivos, balance dedicado por wallet, recargas, retiros, solicitudes de pago, búsqueda de destinatarios, y el CRUD completo de tarjetas y notificaciones. Estas quedaron documentadas para construirse en modo mock en el frontend hasta que se decida si se implementan en este backend.

4. Fase 2 — Mejoras aplicadas al backend

A partir de la auditoría se elaboró un plan priorizado (`docs/API_IMPROVEMENTS.md`) que clasificó cada mejora en dos niveles de riesgo: **Tier A** (no toca autenticación, dinero ni cifrado, no rompe el contrato — aplicable sin aprobación adicional) y **Tier B** (toca autenticación, dinero o cifrado, o es potencialmente incompatible hacia atrás — requiere aprobación explícita por bloque antes de tocar código, según la política de riesgo seguida en todo el proyecto).

Tier A — aplicado íntegramente (10 ítems)

- Documentación OpenAPI con springdoc.
- El error 500 genérico dejó de filtrar el mensaje interno de la excepción al cliente.
- Mapeo correcto a 4xx de excepciones de dominio que antes caían en 500 genérico.
- `@Valid` agregado donde faltaba en varios controladores.
- Validación agregada a los DTOs de PIN y de pago QR.
- Se dejó de loguear el código OTP en texto claro.
- Verificación de unicidad antes de persistir IDs generados aleatoriamente.
- Paginación por cursor real en el listado de transacciones (compatible con el `page` / `size` anterior).
- Límite de tasa (rate limiting) en autenticación, transferencias y pago QR.
- Logging estructurado en JSON para producción.

> **Bug de producción preexistente descubierto y corregido:** el proyecto no heredaba la configuración estándar de Spring Boot y a su compilador le faltaba el flag que preserva los nombres de parámetros por reflexión. En la práctica, esto significaba que cualquier endpoint que usara `@PathVariable` / `@RequestParam` sin nombre explícito ya fallaba en producción. Se corrigió nombrando explícitamente los parámetros afectados en seis controladores — una corrección real de un bug existente, no un cambio de contrato.

Tier B — bloques autorizados y aplicados

Bloque Auth

Se reemplazó el `permitAll()` general por autenticación real exigida a nivel de framework, manteniendo explícitamente públicas solo las rutas que debían serlo. Se corrigió el IDOR en la lectura de transacciones y transferencias por ID (ahora valida que el recurso pertenezca al usuario autenticado). Se activó el rol ADMIN real sobre los endpoints administrativos. Durante la implementación se descubrieron y corrigieron dos problemas no listados en la auditoría original: la ausencia de un punto de entrada de autenticación explícito causaba 403 en vez de 401 para peticiones sin token, y los rechazos de rol no tenían un manejador de excepción dedicado y caían en un 500 genérico.

Bloque PIN / Confirmación

El PIN del usuario pasó de texto plano a hash `bcrypt`, con una estrategia de migración oportunista: en el primer ingreso correcto de PIN tras el despliegue, si el valor almacenado todavía está en texto plano, se verifica por compatibilidad y se re-hashea inmediatamente — sin necesidad

de un script de migración por lotes. Se introdujo además un token de confirmación de un solo uso y vida de 5 minutos, emitido al verificar el PIN correctamente y exigido en las dos operaciones de dinero existentes (transferencias y pago QR) mediante la cabecera `X-Confirmation-Token`.

Bloque Dinero / Firestore

Las dos mutaciones de saldo que no usaban transacciones atómicas de Firestore (pago QR y creación suelta de transacciones) se migraron a un método atómico común que relee y revalida el saldo dentro de la transacción, replicando el patrón que ya se usaba correctamente en las transferencias cuenta-a-cuenta. El dinero dejó de persistirse como `double` en Firestore y pasó a almacenarse como `String` decimal exacto, con lectura dual para seguir funcionando correctamente sobre documentos antiguos aún no migrados.

Resultado de verificación

Tras aplicar los diez ítems de Tier A y los tres bloques autorizados de Tier B, la suite completa de tests del backend — incluyendo los tests nuevos escritos para cada bloque y las correcciones a tests preexistentes que fallaban por motivos ajenos a este trabajo — pasa en su totalidad: **98 de 98 tests**. No se ejecutó ningún comando de git durante esta fase; los cambios quedaron en el árbol de trabajo del backend para revisión del desarrollador, sin tocar los archivos que ya tenían cambios sin confirmar previos a este trabajo.

Quedaron explícitamente fuera de alcance en esta fase, pendientes de autorización futura: el formato de error RFC 9457 Problem Details (cambia la forma de la respuesta de error, rompe contrato), la separación de perfiles CORS dev/prod, el cierre de los endpoints de siembra/purga de base de datos, y la validación completa del contenido del request en la idempotencia.

5. Infraestructura y control de versiones

El repositorio del frontend (`NexaBank-Frontend`) no existía al inicio de este trabajo. Se inicializó como repositorio Git local y se creó como repositorio público en GitHub bajo la cuenta del desarrollador, con la rama por defecto `main`. Se configuró un `.gitignore` cubriendo artefactos de build, variables de entorno reales, archivos locales de configuración del asistente, y la carpeta de desarrollo del service worker PWA.

El repositorio del backend (`NexaBank-Backend`) ya existía y tenía cambios sin confirmar propios del desarrollador al momento de iniciar este trabajo; se verificó explícitamente, en cada entrega sobre ese repositorio, que esos archivos no fueran tocados ni se ejecutara ningún comando de git (`commit`, `add`, `stash`) sobre él — todos los cambios de la Fase 2 quedaron como modificaciones sin confirmar, a la espera de revisión del desarrollador.

> El primer commit y push del frontend al repositorio remoto se realizó directamente por el desarrollador desde su editor durante el desarrollo de este trabajo, en paralelo a la asistencia de este informe — una muestra de colaboración simultánea entre el desarrollador y el proceso de desarrollo asistido.

6. Fase 4-5 — Scaffold y primer sistema de diseño del frontend

Se construyó el frontend desde cero como una SPA/PWA mobile-first, siguiendo el stack definido en la especificación maestra del proyecto y verificando la disponibilidad real de cada versión antes de instalarla.

Stack técnico instalado

Área	Tecnología
Lenguaje / build	TypeScript (modo estricto) · Vite
UI	React 19
Ruteo	React Router v7 (modo data/SPA)
Estado servidor / cliente	TanStack Query v5 · Zustand
Estilos	Tailwind CSS v4 (config CSS-first con `@theme`) + variables CSS para tokens
Primitivas UI accesibles	Radix UI (Dialog, Tabs)
Iconografía	lucide-react
PWA	vite-plugin-pwa (Workbox) — app shell instalable, manifest, service worker

Se definió una estructura de carpetas por feature (`app/`, `features/`, `shared/`) separando componentes de dominio de la capa reutilizable del sistema de diseño.

Primer sistema de diseño (reemplazado posteriormente — ver sección 8)

La primera iteración del sistema de diseño implementó una paleta clara con acento morado, tipografía Inter, y un conjunto base de componentes (`Button`, `Input`, `Card`, `BalanceCard`, `TabBar`, `TransactionItem`, `BottomSheet`, entre otros), todos revisables en una ruta dedicada `/dev/ui` construida como alternativa ligera a Storybook. Sobre esta base se construyeron las primeras pantallas navegables con datos simulados: Home (saldo, acciones rápidas, contactos frecuentes, últimos movimientos), una pantalla de acceso, y el flujo completo de envío de dinero (selección de contacto → monto → confirmación → resultado), con un store de Zustand que de verdad descuenta el saldo simulado y agrega el movimiento al historial — el criterio de calidad exigido fue que un mock "se sintiera real" al navegarlo, no solo mostrar pantallas estáticas.

Durante esta fase se detectó y documentó un hallazgo de accesibilidad: uno de los pares de color texto/fondo de la paleta original no alcanzaba el contraste mínimo WCAG AA. Como los valores de color habían sido especificados de forma exacta, no se alteraron unilateralmente — se mitigó limitando dónde se usaba ese tono, y se dejó documentada la recomendación de ajuste para una fase posterior. El hallazgo terminó resolviéndose de raíz en el rediseño descrito en la sección 8.

7. Corrección del flujo de autenticación

La primera versión de la pantalla de acceso usaba el patrón genérico de correo y contraseña. Se identificó que esto no correspondía al modelo real del backend auditado, que autentica por **número de teléfono + código OTP (primera vez) + PIN o biometría (usuarios recurrentes)** — el patrón típico de billeteras móviles latinoamericanas, donde el correo es información complementaria de verificación de cuenta, nunca una credencial de acceso.

Se reconstruyó el flujo completo de autenticación, totalmente simulado (sin llamadas reales al backend todavía), compuesto por cinco pantallas nuevas:

Pantalla	Función
`/auth/phone`	Ingreso del número de teléfono; punto de entrada de la app
`/auth/otp`	Verificación por código de 6 dígitos con auto-avance, pegado de código y reenvío simulado
`/auth/register`	Datos complementarios (nombre, correo opcional) para teléfonos nuevos — el texto en pantalla aclara explícitamente que el correo es para verificación, no para iniciar sesión
`/auth/create-pin`	Creación de PIN de 4 dígitos con doble confirmación
`/auth/pin`	Pantalla de desbloqueo para dispositivos ya reconocidos: avatar, nombre, teléfono, teclado numérico y un botón prominente de biometría ("Usar Face ID")

El teclado numérico, construido originalmente dentro del flujo de transferencia, se extrajo a un componente reutilizable y ahora se comparte entre el ingreso de monto y el ingreso de PIN. Se construyó también un componente de entrada de código OTP de seis casillas con auto-avance y soporte de pegado.

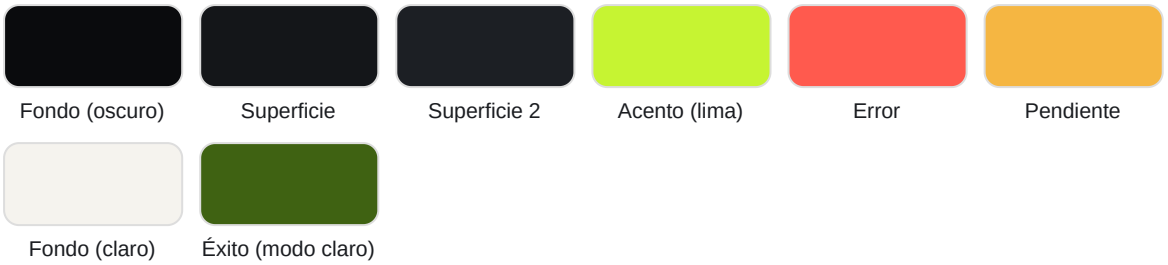
El estado de "cuenta registrada" y "dispositivo reconocido" se simula con un store persistido indexado por número de teléfono (no un flag global), de forma deliberada: esto permite probar en la misma sesión del navegador tanto el camino de un usuario nuevo (teléfono → OTP → registro → creación de PIN) como el de un usuario recurrente (teléfono reconocido → desbloqueo directo con PIN o biometría), sin que probar uno interfiera con el otro.

> El botón de biometría es completamente visual — simula un escaneo de ~1.2 segundos y navega incondicionalmente al inicio; no implementa WebAuthn real. Esto queda documentado explícitamente como pendiente para cuando exista integración real con el backend.

8. Rediseño "Obsidiana y Lima"

Tras validar los flujos funcionales con el primer sistema de diseño, se reemplazó íntegramente la identidad visual por un concepto oscuro-primero definido explícitamente por el desarrollador: **"Obsidiana y Lima"** — superficies oscuras sobrias y un único color de acento muy vivo, en la línea estética de fintechs como Revolut Metal o Nubank Ultravioleta: poco color, mucha jerarquía. El cambio se aplicó a **todas** las pantallas ya construidas, no solo a componentes nuevos.

Paleta de color



El modo oscuro es ahora el **valor por defecto** de la aplicación (antes seguía la preferencia del sistema operativo); el modo claro queda como una elección manual explícita, reservada para una futura pantalla de Ajustes. Para evitar un parpadeo del tema incorrecto antes de que React monte, el documento HTML fija el tema oscuro de forma estática desde el primer byte.

Regla de uso estricta: el color de acento lima cubre como máximo un 10% de la pantalla — botón principal, montos entrantes, tab activo y el botón central de escaneo QR — y no se usan gradientes en ningún lugar; la profundidad visual viene de superficies escalonadas y bordes finos de 1px.

Tipografía

Se adoptó **Geist** (interfaz) y **Geist Mono** (referencias y códigos), autohospedadas vía fontsource, tras confirmar su disponibilidad real en el registro de npm — se descartó la necesidad del fallback Manrope previsto originalmente. El saldo principal se renderiza en dos niveles: la parte entera en tamaño grande (44px, peso 600, tracking negativo) y los decimales a un 60% de ese tamaño en gris, mediante un componente dedicado que separa el valor ya formateado sin asumir un separador decimal específico.

Componentes y patrones clave

- **Tarjeta de saldo:** superficie oscura plana (sin gradiente), textura de ruido sutil al 3% generada con un filtro SVG, borde de 1px en lima translúcido, etiqueta "SALDO DISPONIBLE" y línea de variación mensual en lima.
- **Botones y chips:** forma de píldora en todas las variantes; el botón primario usa relleno lima con texto negro a 56px de alto.
- **Barra de navegación inferior:** flotante, separada del borde, con fondo translúcido y desenfoque de fondo (`backdrop-filter`); el botón central de escaneo QR es un círculo lima elevado sobre la línea base, como un botón de acción flotante.
- **Lista de transacciones:** sin divisores entre filas, separación únicamente por espaciado.

- **Movimiento:** conteo ascendente del saldo al entrar a Home (600ms), transición horizontal entre pasos del flujo de envío (200ms), entrada tipo resorte en los sheets, y en la pantalla de éxito de una transferencia, un checkmark que se dibuja en lima acompañado de una vibración breve y la animación de "aterrizaje" del monto final. Todas las animaciones respetan la preferencia de movimiento reducido del sistema operativo, verificada tanto a nivel CSS como explícitamente en JavaScript para las animaciones controladas por código.

Decisiones de diseño no especificadas explícitamente

Varios valores no estaban definidos en el concepto original y se derivaron con criterio propio, verificando contraste WCAG AA en cada caso (fórmula de contraste relativo estándar): los fondos translúcidos de las insignias de estado (`Badge`), un color "informativo" neutro que no compite con el lima como segundo acento, y la versión clara del fondo translúcido de la barra de navegación. Se eliminó por completo un tercer nivel de texto secundario que arrastraba el problema de contraste detectado en la iteración anterior, resolviéndolo de raíz en vez de solo mitigarlo.

9. Verificación y aseguramiento de calidad

En cada entrega de frontend se siguió el mismo protocolo de verificación, sin excepciones:

1. Compilación estricta de TypeScript y build de producción (`npm run build`) sin errores ni uso de `any`.
2. Arranque real del servidor de desarrollo y comprobación por HTTP de que cada ruta de la aplicación responde correctamente.
3. **Navegación real con un navegador Chromium headless** (Playwright) sobre cada pantalla clave, en modo oscuro y en modo claro, capturando evidencia visual y verificando la ausencia de errores de consola o de página — en lugar de confiar en el reporte textual de la construcción del código.
4. Revisión manual de las capturas resultantes contra el criterio de diseño original antes de informar la fase como terminada.

En el backend, la verificación se realizó ejecutando la suite de tests de Maven de punta a punta tras cada bloque de cambios, nunca solo compilando. Esto permitió descubrir, además de los defectos originales de la auditoría, varios bugs adicionales no relacionados con el alcance de la tarea (fallas en tests preexistentes, un bug de compilación que rompía el enlace de parámetros HTTP en producción) que se corrigieron y documentaron por separado de las decisiones de producto.

> Ningún cambio se reportó como completo sin evidencia observable adjunta: comando ejecutado y resultado obtenido, o captura de pantalla de la interfaz real funcionando.

10. Estado actual y trabajo pendiente

Completo y verificado

- Auditoría completa de la API backend, con 19 riesgos clasificados y documentados.
- Diez mejoras de bajo riesgo aplicadas al backend, más tres bloques de alto riesgo (autorización real, hash de PIN + token de confirmación, transacciones atómicas de dinero), con 98/98 tests pasando.
- Repositorio de frontend creado, configurado y publicado en GitHub.
- Scaffold completo de la PWA: build de producción funcional, service worker, manifest instalable.
- Sistema de diseño "Obsidiana y Lima" aplicado de punta a punta sobre todas las pantallas existentes.
- Flujo de autenticación por teléfono + OTP + PIN/biometría, completamente navegable en modo simulado.
- Flujo completo de envío de dinero, con estado mock que refleja cambios reales de saldo e historial.

Pendiente (según las fases originales de la especificación)

- **Fase 3:** generación del contrato OpenAPI real y cliente tipado generado a partir de él (actualmente los tipos de dominio están escritos a mano, modelados para parecerse al contrato esperado).
- **Fase 6:** capa de mocks con Mock Service Worker interceptando peticiones de red reales (hoy los datos simulados viven directamente en memoria vía Zustand).
- **Fase 7:** pantallas restantes — actividad/historial completo, tarjetas, perfil, ajustes, KYC, notificaciones, recibir/recargar/retirar, escaneo real de QR — actualmente son pantallas de "Próximamente".
- **Fase 8:** suite de tests automatizados (Vitest, Testing Library, Playwright E2E), auditoría de accesibilidad formal y medición de Lighthouse, README de instalación.
- Bloques de Tier B del backend aún no autorizados: formato de error estándar RFC 9457, separación de perfiles CORS, cierre de los endpoints de siembra/purga, validación completa de idempotencia.

11. Conclusiones

El trabajo realizado hasta este punto cumple el principio rector de la especificación original: no se construyó frontend sobre suposiciones acerca de una API sin verificar, y una vez verificada, el frontend se construyó de forma completa y navegable — aunque sea en modo simulado — en lugar de quedarse en diagramas o componentes aislados sin integrar entre sí.

La auditoría temprana del backend resultó determinante: reveló que la superficie de autenticación real del backend (teléfono + OTP + PIN) era distinta de lo que inicialmente se había construido en el frontend (correo + contraseña), y permitió corregirlo antes de que esa discrepancia se propagara a más pantallas. De igual forma, tratar explícitamente como alto riesgo cualquier cambio de autenticación, dinero o cifrado — exigiendo aprobación por bloque en vez de aplicar todo de una vez — permitió avanzar con velocidad en las partes seguras del trabajo (documentación, validación, paginación, logging) sin sacrificar control sobre las partes sensibles (hash de PIN, transacciones atómicas de dinero, modelo de autorización).

El proyecto queda en un estado sólido para continuar: backend más seguro y con cobertura de tests real, y un frontend con una identidad visual definida, un flujo de autenticación correcto, y al menos un flujo de dinero de punta a punta funcionando de forma creíble en modo mock — la base necesaria para conectar el contrato OpenAPI real y el resto de las pantallas en las fases siguientes.